

Projet FPGA – Sécurité

Chiffrement ASCON-128 sur FPGA PynqZ2
pour acquisition sécurisée de signaux ECG

Ali AHNANI | **Khalid ELKOUSSAMI**

Encadrants : Jean-Baptiste Rigaud & Raphaël Viera

École des Mines de Saint-Étienne — Institut Mines-Télécom 2025–2026

Contexte & Objectifs

Problematique

Signaux ECG = donnees medicales sensibles

Risques : **vol** et **falsification** sur liaison serie non protegee

⇒ Assurer **confidentialite** + **authenticite** en temps reel

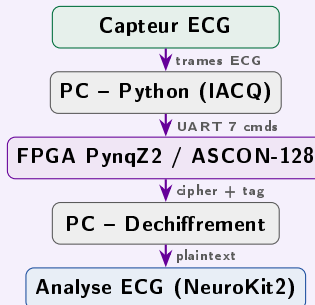
Choix : ASCON-128 (AEAD)

- Standard NIST 2023, conçu pour **embarque** / **IoT**
- Chiffrement + authentification en un seul passage
- Faible empreinte sur FPGA

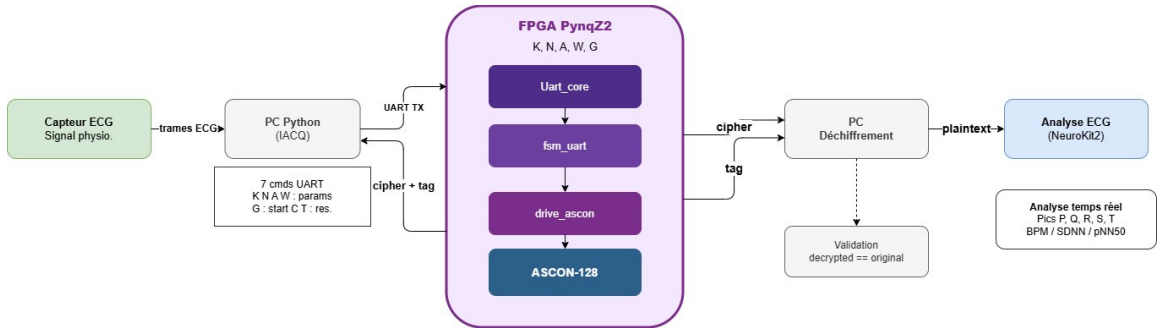
Parametres ASCON-128

Cle 128b	Nonce 128b	AD 64b	Tag 128b
Trame 181 oct.	UART 115 200 baud		

Flux de donnees



Synoptique du Projet



ascon_top : Vue Externe



Interface ascon_top

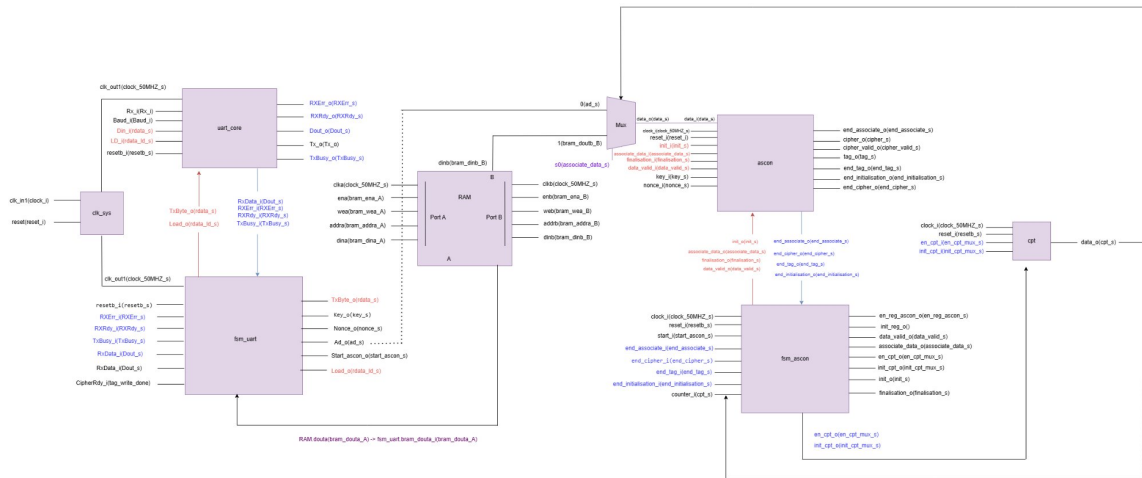
Entrees :

- clock_i – Horloge systeme
- reset_i – Reset synchrone
- Rx_i – Reception UART
- Baud_i[2:0] – Baud rate

Sorties :

- Tx_o – Emission UART
- Baud_o[2:0] – Baud rate sortie
- RTS_o – Ready To Send

Module top-level : chaine complete UART + ASCON-128.



Machine d'Etats fsm_uart : Interface UART

Role

Decodeur UART : recoit commandes, stocke parametres, declenche ASCON, renvoie cipher + tag + "OK\n".

Reception octet par octet

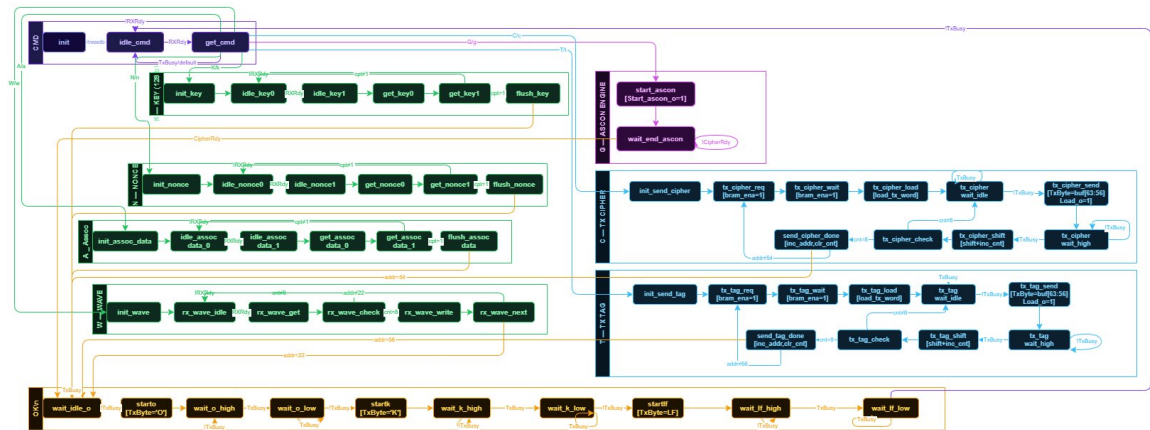
1 octet hex → 8 bits via UART
⇒ stocke dans les registres ou en BRAM

7 commandes

Cmd	Hex	Donnees
K – Cle	0x4B	16 oct.
N – Nonce	0x4E	16 oct.
A – Assoc. Data	0x41	8 oct.
W – Waveform	0x57	184 oct.
G – Go (start)	0x47	—
C – Cipher	0x43	185 oct.
T – Tag	0x54	17 oct.

Acquittement

Après chaque commande : "OK\n"
starto → sendo → startk → sendk → startlf → sendlf



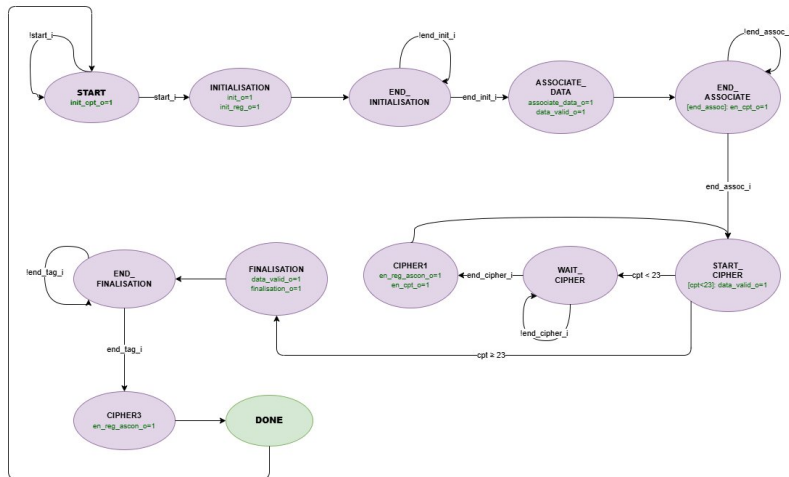
Machine d'Etats drive_ascon

Role

Sequencier des 4 phases ASCON.
Pilote le compteur de blocs et signale a fsm_moore chaque phase.

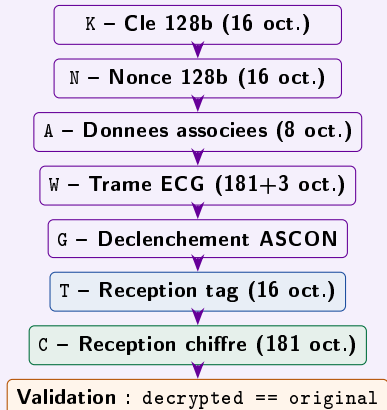
Signaux cls

Signal	Role
start_i	Declenchement
end_init_i	Fin init
end_cipher_i	Fin bloc cipher
end_tag_i	Fin tag
cpt_i[4:0]	Compteur blocs
init_o	Lance init
data_valid_o	Bloc pret
finalisation_o	Lance final.



Note : TEST_CPT = 22 $\Rightarrow$ 22 blocs de 64 bits pour 181 octets ECG.

Flux d'une trame ECG



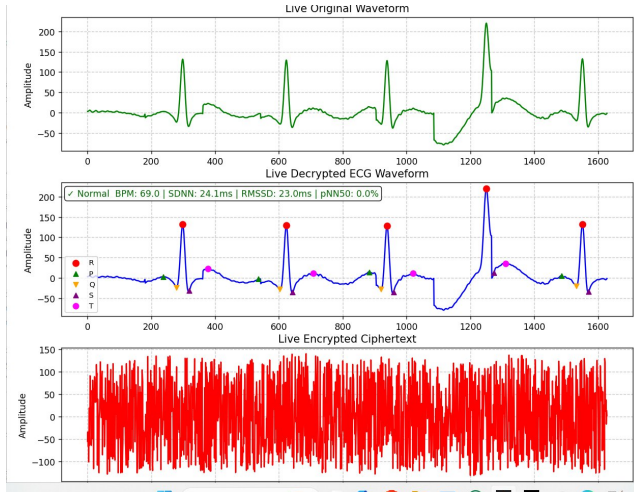
Modules Python

Fichier	Role
iacq.py	Classe IACQ : pyserial, retry x3
ascon_pcsn.py	Dechiffrement ASCON-128 ref.
fpga_emulator.py	Emulateur sans FPGA
visualization.py	3 courbes temps reel
main.py	Orchestration + NeuroKit2

Analyse ECG (NeuroKit2)

Filtrage HPF 0.5 Hz + LPF 50 Hz
Detection pics P, Q, R, S, T
HRV : BPM, SDNN, RMSSD, pNN50

Resultats : Validation Experimentale



Original (vert), ECG dechiffre + PQRST (bleu), chiffre (rouge)

Resultats

- ✓ Validation **bit-a-bit** : decrypted == original
- ✓ Tag ASCON-128 verifie (128 bits)
- ✓ Signal ECG parfaitement recupere
- ✓ BPM = 69.0 (sinusal normal)
SDNN = 24.1 ms RMSSD = 23.0 ms
- ✓ Statut : **Normal**

Observation

Signal chiffre (rouge) = **visuellement aleatoire**
⇒ aucune info extractible sans la cle.

Mise au Point : Erreurs et Débug

Erreur	Symptôme	Correction
Délais UART en real sous XSim	Rx_i semble fixe, RXRdy_s=0	Constantes entières : BIT_NS=8960
Horloge TB 50 MHz au lieu de 125 MHz	MMCM ne locke pas, logique interne gelée	clock_i à 125 MHz dans le TB
Stimulation avant lock PLL	Premières commandes perdues	Attente 2000+ cycles après reset
Mauvais zoom waveform	Faux diagnostic "Rx_i bloqué"	Zoom sur timestamps console
Variables locales XSim	Comportement erratique	Variables au niveau module
Pas d'instrumentation RX	Impossible de vérifier les ACK	Buffer tx_log[], tx_wp, wait_tx()
Latence BRAM (registre sortie)	Cipher/tag décalés d'un mot	Cycle d'attente supplémentaire dans la FSM

Méthode clé : console (\$display) + waveform en corrélation. La console dit ce que le TB *croit* faire ; la waveform dit ce que le DUT *fait réellement*. Le débbug consiste à comparer les deux, en zoomant sur les timestamps console.

Conclusion & Perspectives

Réalisations

- ✓ Architecture complète `ascon_top` intégrant UART, BRAM, ASCON-128 et 3 FSM
- ✓ Protocole 7 commandes UART validé par testbench auto-vérifiant (0 échec)
- ✓ Chiffrement ASCON-128 fonctionnel : cipher + tag corrects bit-à-bit
- ✓ Chaîne Python IACQ + déchiffrement + analyse ECG (NeuroKit2) en temps réel
- ✓ Validation end-to-end : `decrypted == original`, BPM = 69.0 (normal)

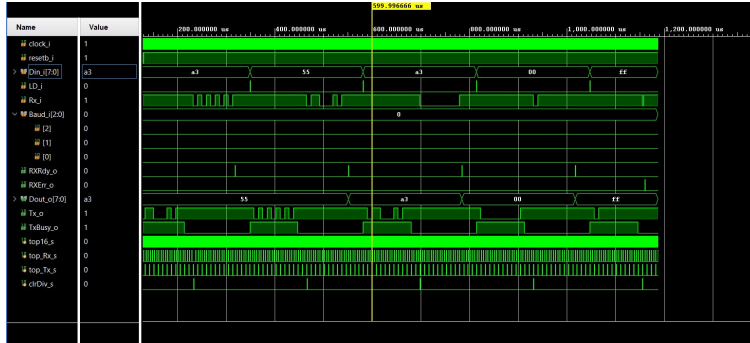
Perspectives

- Intégration sur carte PynqZ2 avec capteur ECG physique
- Pipeline ASCON pour augmenter le débit
- Transmission multi-frames en continu
- Tests de sécurité : DPA / SCA
- Extension à d'autres capteurs biomédicaux

Ali AHNANI & Khalid ELKOUSSAMI | Encadrants : J.-B. Rigaud & R. Viera

École des Mines de Saint-Étienne — Institut Mines-Télécom

Annexe 1 : Testbench uart_core



Chronogramme de simulation — validation fonctionnelle de `uart_core`

Validation

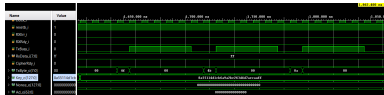
- ✓ Octets envoyés sur Rx_i correctement reconstruits sur Dout_o
- ✓ RXRdy_o pulse à chaque réception valide
- ✓ RXErr_o reste nul
- ✓ Transmission confirmée par Tx_o et TxBusy_o

Conclusion

Le bloc `uart_core` est valide en **reception** et en **emission**.

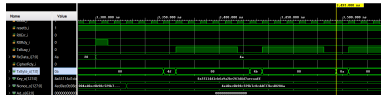
Annexe 2 : Testbench fsm_uart — Cmds K, N, A

K – Cle 128b



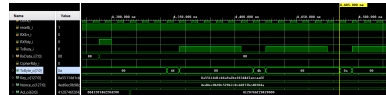
✓ Key_o = 8a55...aaff

N – Nonce 128b



✓ Nonce_o = 4ed0...284a

A – Assoc. Data 64b

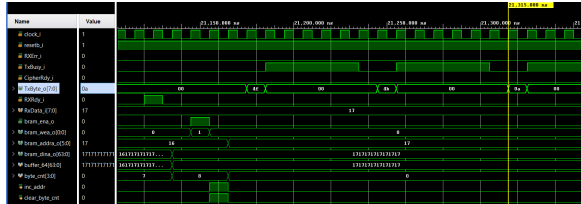


✓ Ad_o = 4120...8000

Pour chaque commande : les octets sont correctement assembles dans les registres, et l'ACK "0K\n" (4F 4B 0A) est retransmis.

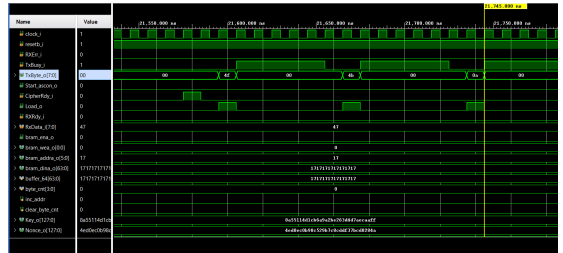
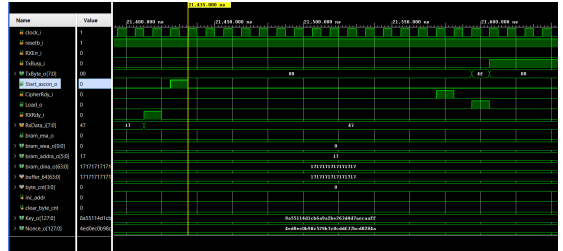
Annexe 2 : Testbench fsm_uart — Cmds W, G

W – Wave 23×64b → BRAM[0..22]



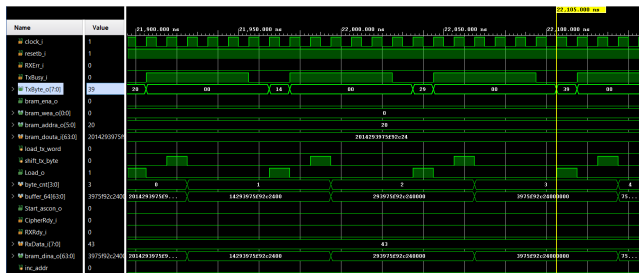
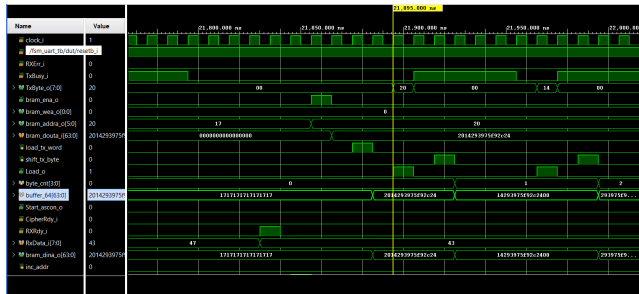
✓ 23 mots écrits, bram_wea_o pulse, adresses correctes

G – Lancement ASCON



✓ Start_ascon_o pulse, attente CipherRdy

Annexe 2 : Testbench fsm_uart — Cmd C (Cipher)



C – Cipher 184 octets

Prechargement mem[32..54] avec 23 mots cipher de reference.

La FSM lit chaque mot via BRAM (latence 1 cycle), puis emet 8 octets MSB-first.

Validation console

✓ 184 octets retransmis fidelement

✓ Ordre MSB-first correct

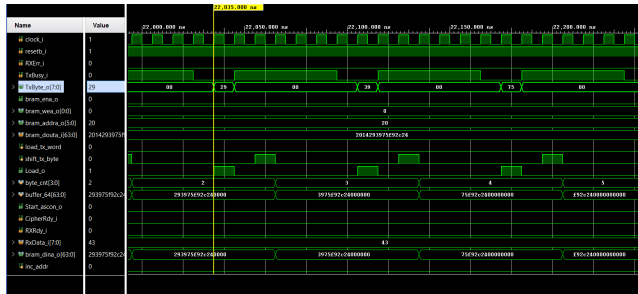
Ex: mot 0 → 20 14 29 39 75 F9 2C 24

= 2014293975F92C24

✓ ACK "0K\n" apres donnees

tx_log[184..186] = 4F 4B 0A

Annexe 2 : Testbench fsm_uart — Cmd T (Tag)



Sortie console (extrait)

```
[35525000 ns] TX = e2
[35595000 ns] TX = 39
[35665000 ns] TX = 31
[35735000 ns] TX = 73
[35805000 ns] TX = 39
[35875000 ns] TX = 02
[35945000 ns] TX = c7
[36015000 ns] TX = ca
[36105000 ns] TX = 08
[36665000 ns] TX = 4f
[36735000 ns] TX = 4b
[36805000 ns] TX = 0a
```

T – Tag 16 octets

Lecture de mem[55] et mem[56] (2 mots de 64 bits).

Le TB compare octet par octet :

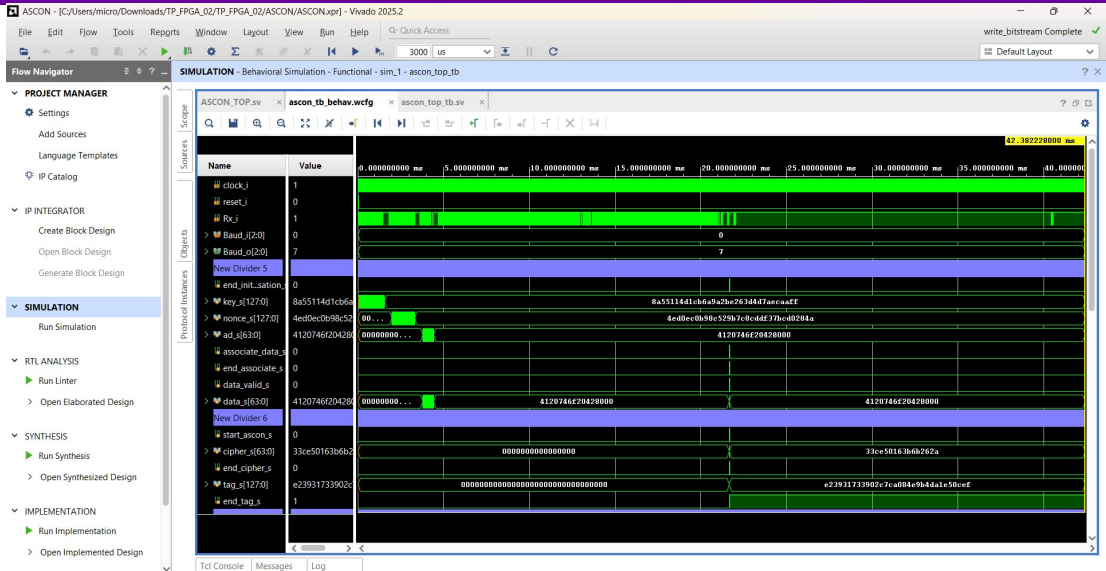
```
got_b = tx_log[wp_base + i*8 + j]
```

```
exp_b = tag_ref[i] [(7-j)*8 +: 8]
```

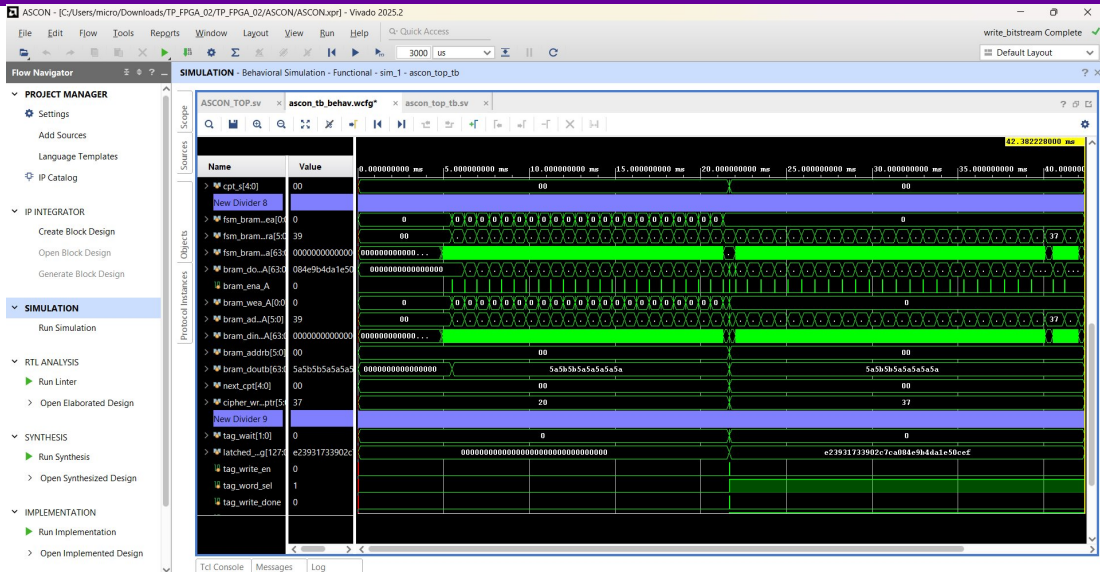
Validation

- ✓ tag_ref[0] = E239...C7CA
- ✓ tag_ref[1] = 084E...0CEF
- ✓ 16 octets corrects MSB-first
- ✓ ACK 4F 4B 0A apres tag
- ✓ wait_tx(wp_base + 19)
- (16 data + 3 ACK = 19 pulses Load_o)

Annexe 3 : Testbench ascon_top — Signaux top-level



Annexe 3 : Testbench ascon_top — Signaux internes



Annexe 4 : ASCON_TOP.sv — Module top-level

Rôle général

Ce module est le top-level de notre design. Il instancie et connecte tous les blocs du projet : le générateur d'horloge `clk_sys`, le bloc `uart_core` pour la communication série, la FSM `fsm_uart` qui décode les commandes, le cœur cryptographique `ascon`, le séquenceur `fsm_ascon`, le compteur de blocs `cpt`, et une BRAM dual-port qui sert de mémoire partagée.

Arbitrage du Port A de la BRAM

Le port A de la BRAM est partagé entre trois sources d'écriture. On a mis en place un arbitre avec une priorité fixe : l'écriture du tag est la plus prioritaire, puis l'écriture du cipher, et enfin les accès de la FSM UART. Cela évite toute collision sur le port d'écriture.

Organisation de la BRAM

La mémoire est divisée en trois zones : les adresses 0 à 22 stockent les 23 mots de la wave ECG, les adresses 32 à 54 reçoivent les 23 mots du cipher après chiffrement, et les adresses 55 et 56 contiennent les deux mots du tag de 128 bits.

Écriture du cipher et du tag

Pour le cipher, un pointeur `cipher_write_ptr` démarre à 32 et s'incrémente à chaque impulsion de `en_reg_ascon_s`. Pour le tag, on utilise un double registre anti-glitch sur `end_tag_s`, suivi d'un latch immédiat de la valeur, puis une ligne à retard de 2 cycles avant d'écrire les deux mots en BRAM. Un sticky bit `ascon_done_flag` signale à la FSM UART que le calcul est terminé.

Annexe 4 : ASCON_TOP.sv — Extraits de code

Arbitre Port A (TAG > CIPHER > UART)

```
always_comb begin
    ascon_req = 1'b0;
    if (tag_write_en) begin
        ascon_req = 1'b1;
        ascon_addr = tag_word_sel
            ? 6'd56 : 6'd55;
        ascon_data = tag_word_sel
            ? latched_tag[63:0]
            : latched_tag[127:64];
    end else if (en_reg_ascon_s
        && cipher_write_ptr <= 6'd54)
    begin
        ascon_req = 1'b1;
        ascon_addr = cipher_write_ptr;
        ascon_data = cipher_s;
    end
end
assign bram_ena_A = ascon_req
    | fsm_bram_ena;
assign bram_wea_A = ascon_req
    ? 1'b1 : fsm_bram_wea;
```

Double registre anti-glitch + latch tag

```
// 1. Anti-glitch
{tag_r1, tag_r2}
    <= {end_tag_s, tag_r1};
assign safe_tag_rising
    = tag_r1 & ~tag_r2;

// 2. Latch immediat
if (safe_tag_rising)
    latched_tag <= tag_s;

// 3. Delay line 2 cycles
tag_wait
    <= {tag_wait[0],
        safe_tag_rising};
```

Sticky bit (handshake ASCON→UART)

```
if (start_ascon_s)
    ascon_done_flag <= 1'b0;
else if (tag_write_done)
    ascon_done_flag <= 1'b1;
```

Annexe 5 : fsm_UART.sv — FSM Interface UART

Rôle et structure générale

Cette FSM est le décodeur du protocole UART. Elle reçoit les commandes envoyées par le PC, stocke les paramètres dans les registres à décalage, écrit la wave en BRAM, déclenche le chiffrement ASCON, puis relit le cipher et le tag depuis la BRAM pour les retransmettre au PC. Après chaque commande, elle envoie un acquittement "OK\n". Elle comporte environ 60 états organisés en 8 groupes : dispatch des commandes, réception K/N/A, écriture wave, lancement ASCON, envoi cipher, envoi tag, et acquittement.

Réception des données

Pour les commandes K, N et A, chaque octet reçu est poussé dans un registre à décalage dédié. Pour la wave, les octets sont accumulés par groupes de 8 dans un buffer de 64 bits, puis écrits en BRAM à chaque mot complet.

Sous-modules instanciés

La FSM instancie cinq sous-modules : `key_reg_shift` pour assembler la clé de 128 bits, `reg_shift` pour le nonce de 128 bits, `a_data_reg_shift` pour les données associées de 64 bits, `main_decounter` qui est un décompte capable de gérer plusieurs longueurs selon la commande, et `trans_receive` qui convertit les octets reçus.

Retransmission du cipher et du tag

Pour les commandes C et T, la FSM lit la BRAM mot par mot, charge chaque mot de 64 bits dans un buffer, puis le décale octet par octet vers le transmetteur UART via le signal `Load_o`. Une fois toutes les données envoyées, elle transmet l'acquittement 4F 4B 0A (c'est-à-dire O, K, saut de ligne).

Annexe 5 : fsm_UART.sv — Extraits de code

Buffer 64 bits et gestion adresses

```
// Accumulation des octets recus
if (load_rx_byte)
    buffer_64
        <= {buffer_64[55:0],
            data_converted_s};

// Chargement mot BRAM pour TX
else if (load_tx_word)
    buffer_64 <= bram_douta_i;

// Decalage octet par octet
else if (shift_tx_byte)
    buffer_64
        <= {buffer_64[55:0], 8'h00};

// Adresses selon la commande
if (set_addr_wave)
    bram_addr_a_s <= 6'd0;
else if (set_addr_cipher)
    bram_addr_a_s <= 6'd32;
else if (set_addr_tag)
    bram_addr_a_s <= 6'd55;
else if (inc_addr)
    bram_addr_a_s
```

États wave : réception → BRAM

```
rx_wave_idle:
    etat_f = (RXRdy_i)
        ? rx_wave_get
        : rx_wave_idle;

rx_wave_check:
    etat_f = (byte_cnt == 4'd8)
        ? rx_wave_write
        : rx_wave_idle;
```

Acquittement OK\n

```
// Envoi "0" puis "K" puis LF
starto: TxByte_o = 8'h4F;
        Load_o  = 1'b1;
startk: TxByte_o = 8'h4B;
        Load_o  = 1'b1;
startlf: TxByte_o = 8'h0A;
        Load_o  = 1'b1;
```

Chaque envoi attend TxBusv_i haut puis bas avant de

Annexe 6 : drive_ascon.sv — Séquenceur ASCON

Rôle

Ce module est le séquenceur de haut niveau qui orchestre les quatre phases du chiffrement ASCON. Il pilote le compteur de blocs et indique au cœur cryptographique `fsm_moore` quand lancer chaque phase. Il possède 12 états, de `START` jusqu'à `DONE`.

Signal critique : `en_reg_ascon_o`

Ce signal est une impulsion d'un seul cycle d'horloge, générée dans les états `CIPHER1` et `CIPHER3`. C'est lui qui déclenche l'écriture de chaque mot cipher dans la BRAM au niveau du module `ASCON_TOP`. Sans cette impulsion précise, le cipher ne serait pas stocké correctement.

Déroulement des états

Le séquenceur commence par réinitialiser le compteur dans `START`, puis lance l'initialisation d'ASCON avec p^{12} . Ensuite, il traite les données associées. Après cela, il entre dans la boucle de chiffrement : tant que le compteur est inférieur à 23, il boucle entre `START_CIPHER`, `WAIT_CIPHER` et `CIPHER1`. Quand les 22 blocs normaux sont traités, il passe en `FINALISATION` pour la permutation finale p^{12} . L'état `CIPHER3` écrit le dernier bloc avec padding, puis `DONE` signale la fin.

Boucle cipher

La condition de boucle est `counter_i < 23`. Quand le compteur atteint 23, on sort de la boucle vers la finalisation. Au total, on traite 22 blocs normaux de 64 bits plus un dernier bloc avec padding dans `CIPHER3`.

Annexe 6 : drive_ascon.sv — Extraits de code

Transitions (next_state)

```
unique case (current_state)
  START:
    if (start_i)
      next_state = INITIALISATION;
  END_INITIALISATION:
    if (end_initialisation_i)
      next_state = ASSOCIATE_DATA;
  END_ASSOCIATE:
    if (end_associate_i)
      next_state = START_CIPHER;

  // Boucle cipher : 22 blocs
  START_CIPHER:
    if (counter_i < 5'd23)
      next_state = WAIT_CIPHER;
    else
      next_state = FINALISATION;

  WAIT_CIPHER:
    if (end_cipher_i)
      next_state = CIPHER1;
  CIPHER1:
    next_state = START_CIPHER;

  END_FINALISATION:
    if (end_tag_i)
      next_state = CIPHER3;
  CIPHER3: next_state = DONE;
  DONE:   next_state = START;
endcase
```

Sorties par état

```
// Reinitialise le compteur
START:
  init_cpt_o = 1'b1;

// Lance la permutation init
INITIALISATION:
  init_o      = 1'b1;
  init_reg_o = 1'b1;

// Active les donnees associees
ASSOCIATE_DATA:
  associate_data_o = 1'b1;
  data_valid_o     = 1'b1;

// Impulsion 1 cycle : ecriture
// du cipher en BRAM
CIPHER1:
  en_reg_ascon_o = 1'b1;
  en_cpt_o       = 1'b1;

// Lance la finalisation p12
FINALISATION:
  data_valid_o  = 1'b1;
  finalisation_o = 1'b1;

// Dernier bloc (padding)
CIPHER3:
  en_reg_ascon_o = 1'b1;
```

Annexe 7 : ascon.sv + Permutation.sv — Datapath ASCON

ascon.sv — Module enveloppe

Ce module est l'enveloppe du cœur cryptographique. Il instancie deux sous-modules : `fsm_moore`, qui est l'automate interne à 25 états gérant les quatre phases du chiffrement, et `Permutation`, qui contient le datapath complet de la permutation ASCON. La FSM `fsm_moore` génère tous les signaux de contrôle : les enables, les sélections de multiplexeur, et le compteur de rondes. Les transitions se font quand le compteur de ronde atteint 10, ce qui correspond à la fin d'une permutation.

Les 4 phases de fsm_moore

L'initialisation utilise p^{12} (12 rondes), l'association des données utilise p^6 (6 rondes), le chiffrement répète p^6 pour chacun des 22 blocs, et la finalisation utilise à nouveau p^{12} pour générer le tag.

Permutation.sv — Chaîne de calcul

Ce module implémente une ronde de la permutation ASCON. La chaîne de calcul est la suivante : d'abord un multiplexeur `mux_state` sélectionne l'état initial ou le retour du registre. Ensuite, le bloc `xor_begin` applique le XOR avec les données ou la clé. Puis les trois couches de la permutation s'enchaînent : `Pc` ajoute la constante de ronde, `Ps` applique la SBox non-linéaire de 5 bits, et `P1` réalise la couche de diffusion linéaire. Enfin, `xor_end` applique le XOR final avec la clé, et le résultat est stocké dans le registre d'état.

Sorties du datapath

Le cipher est extrait juste après le `xor_begin`, c'est le MSB de l'état XORé avec le plaintext. Le tag est construit à partir des deux derniers mots de l'état après le `xor_end` final, ce qui donne 128 bits. L'état interne complet fait 5 fois 64 bits, soit 320 bits.

Chaîne d'instanciation complète

```
// 1. Multiplexeur d'entree
mux_state mux_instance (
    .sel_i (select_i),
    .data1_i(state_i),          // init
    .data2_i(output_state_register_s),
    .data_o (output_mux_s));

// 2. XOR entree (data + cle)
xor_begin_perm xor_begin (
    .data_i (data_i),
    .key_i (key_i),
    .state_i(output_mux_s),
    .state_o(output_xor_begin_s));

// 3. Extraction du cipher
register_w_en #(64) UCipher (
    .en_i (enable_cipher_i),
    .data_i(output_xor_begin_s[0]),
    .data_o(cipher_o));
```

Permutation Pc → Ps → Pl + XOR fin

```
// 4. Constante de ronde
Pc Pc_instance (
    .Pc_in_i (output_xor_begin_s),
    .Round_i (round_i),
    .Pc_out_o(output_pc_s));

// 5. SBox 5 bits (non-linéaire)
Ps Ps_instance (
    .Ps_in_i (output_pc_s),
    .Ps_out_o(output_ps_s));

// 6. Couche lineaire
Pl Pl_instance (
    .Pl_in_i (output_ps_s),
    .Pl_out_o(output_pl_s));

// 7. XOR fin + registre d'etat
xor_end_perm xor_end (
    .key_i (key_i),
    .state_i(output_pl_s),
    .state_o(output_xor_end_s));

// 8. Extraction du tag (128b)
register_w_en #(128) UTag (
    .en_i (enable_tag_i),
    .data_i({output_xor_end_s[3],
             output_xor_end_s[4]}),
    .data_o(tag_o));
```

Annexe 8 : Testbenches — Stratégie de validation

ascon_top_tb.sv — TB d'intégration

Ce testbench valide l'ensemble de la chaîne, du signal UART jusqu'au cipher et au tag. Il génère une horloge à 125 MHz pour être compatible avec le MMCM interne. Les délais UART sont définis en constantes entières pour éviter les problèmes XSim. Le TB envoie les trames bit-à-bit sur Rx_i, puis capture les réponses via une tâche capture_uart_byte. Il exécute la séquence complète K, N, A, W, G puis vérifie que le cipher et le tag reçus correspondent aux valeurs de référence.

fsm_uart_tb.sv — TB unitaire

Ce testbench teste la FSM UART de manière isolée, sans passer par le vrai UART ni le MMCM. Il utilise une horloge directe à 100 MHz. La BRAM est modélisée par un simple tableau de 64 mots de 64 bits. Le signal TxBusy est simulé en restant haut pendant 5 cycles après chaque Load_o. Tous les octets émis sont capturés dans un tableau tx_log avec un pointeur tx_wp. Le TB exécute 7 tests auto-vérifiants, un par commande, et compare chaque octet reçu avec les références cipher_ref et tag_ref.

Corrections appliquées

Six corrections ont été nécessaires : l'horloge passée à 125 MHz, les constantes converties en entiers, les littéraux de délai en entier pur, et trois corrections de compatibilité XSim sur les variables et les tasks.

Principe commun aux deux TB

Les deux testbenches utilisent des \$display pour chaque événement, une fonction wait_tx avec timeout pour attendre les réponses, un système PASS/FAIL automatique par test, et un compteur d'erreurs global qui donne le bilan final.

Annexe 8 : Testbenches — Extraits de code

ascon_top_tb : capture UART bit-à-bit

```
localparam integer BIT_NS = 8960;
localparam integer HALF_BIT_NS
    = 4480;

task capture_uart_byte;
    output [7:0] b;
    integer k;
    begin
        @(negedge Tx_o);    // start bit
        #(HALF_BIT_NS + BIT_NS);
        b = 8'h00;
        for (k = 0; k < 8; k=k+1)
            begin
                b[k] = Tx_o;
                if (k < 7) #(BIT_NS);
            end
        #(HALF_BIT_NS);    // stop bit
    end
endtask
```

La tâche détecte le start bit (front descendant de Tx_o), puis échantillonne les 8 bits de données au milieu de chaque période bit.

fsm_uart_tb : envoi octet + simulateur TxBusy

```
task automatic send_byte;
    input [7:0] b;
    begin
        @(negedge clock_i);
        RxData_i = b;
        RXRdy_i = 1'b1;
        @(negedge clock_i);
        RXRdy_i = 1'b0;
        repeat(20) @(negedge clock_i);
    end
endtask

// Simulateur TxBusy
if (Load_o) begin
    TxBusy_i <= 1'b1;
    tx_timer <= 15;
    $display("[%0t] TX => 0x%02h",
        $time, TxByte_o);
end else if (tx_timer > 0) begin
    tx_timer <= tx_timer - 1;
    TxBusy_i <= 1'b1;
end else
    TxBusy_i <= 1'b0;
```

Ici on simule le comportement de l'UART sans instancier le vrai bloc : TxBusy reste haut 15 cycles après chaque Load_o, ce qui laisse le temps à la FSM de

Algorithme ASCON

- [1] C. Dobraunig, M. Eichlseder, F. Mendel, M. Schläffer, *Ascon v1.2 – Submission to NIST LWC*, 2021.
<https://ascon.iaik.tugraz.at/>
- [2] NIST SP 800-232, *Ascon-Based Lightweight Cryptography Standards for Constrained Devices*, août 2025.
<https://csrc.nist.gov/pubs/sp/800/232/final>
- [3] NIST, *Lightweight Cryptography Standardization – Selection of Ascon*, février 2023.
<https://csrc.nist.gov/news/2023/lightweight-cryptography-nist-selects-ascon>

Carte et UART

- [4] TUL, *PYNQ-Z2 Reference Manual*, 2019.
<https://www.tulembedded.com/FPGA/ProductsPYNQ-Z2.html>
- [5] *uart_core* – IP UART fourni par le cours ISMIN, École des Mines de Saint-Étienne.

Simulation et debug Vivado/XSim

- [6] AMD/Xilinx, *Vivado Design Suite User Guide: Logic Simulation (UG900)*, 2023.
<https://docs.amd.com/r/en-US/ug900-vivado-logic-simulation>
- [7] AMD/Xilinx, *Vivado Design Suite User Guide: Programming and Debugging (UG908)*, 2023.
- [8] Xilinx, *Block Memory Generator v8.4 – Product Guide (PG058)*, 2021.
Latence BRAM, registre de sortie, collisions dual-port.

Analyse ECG et Python

- [9] D. Makowski et al., *NeuroKit2: A Python Toolbox for Neurophysiological Signal Processing*, Behavior Research Methods, 2021.
<https://github.com/neuropsychology/NeuroKit>
- [10] *pyserial* – Bibliothèque Python pour la communication série.
<https://pyserial.readthedocs.io/>

Vérification et testbenches

[11] S. Sutherland, *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*, Springer, 3^e éd., 2012.

[12] C. Cummings, *Simulation and Synthesis Techniques for Asynchronous FIFO Design*, SNUG 2002.
Méthodologie de double registre anti-glitch et synchronisation inter-domaines.

[13] C. Cummings, *Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!*, SNUG 2000.
Bonnes pratiques `always_ff` / `always_comb`.

Compatibilité XSim

[14] AMD/Xilinx, *AR#76960: XSim – SystemVerilog Support and Known Limitations*.

Problèmes de variables locales, `tasks automatic`, délais `real`.

[15] IEEE Std 1800-2017, *IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language*.

Stratégies de debug utilisées

[16] *Corrélation console/waveform* : utilisation systématique de `$display` avec timestamps pour piloter le zoom dans les waveforms Vivado. Méthode inspirée de [6] chap. 6 et [11] chap. 9.

[17] *Testbench auto-vérifiant* : buffer de capture TX (`tx_log[]`), pointeur cumulatif `tx_wp`, comparaison automatique PASS/FAIL. Approche inspirée des méthodologies UVM simplifiées [11].

[18] *Diagnostic latence BRAM* : identification du décalage d'un mot via analyse des patterns de sortie cipher/tag. Correction basée sur [8] (registre de sortie activé) et ajout d'un cycle d'attente dans la FSM.

[19] *Anti-glitch par double registre* : technique standard de synchronisation inter-domaines [12], appliquée au signal `end_tag_s` pour éviter les écritures parasites en BRAM.